

Übungsblatt 3: Einführung FreeRTOS und Präemption

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

In dieser Übung vollziehen wir den Wechsel vom Bare-Metal-C-Code zu einem professionellen Echtzeitbetriebssystem (FreeRTOS) auf dem STM32H755 (Cortex-M4). Sie konfigurieren die Toolchain, legen Tasks mit unterschiedlichen Prioritäten an und verifizieren die Konzepte der **Präemption** und der **Starvation** (Verhungern von Tasks) live auf der Hardware.

Aufgabe 1: Toolchain-Setup für FreeRTOS

Bevor wir Code schreiben, müssen wir das Betriebssystem und die Zeitbasis in der STM32CubeIDE konfigurieren.

1. Legen Sie ein neues Projekt für das Board **STM32H755ZI-Q** an.
2. Aktivieren Sie unter *Middleware and Software Packs* FreeRTOS für den Cortex-M4. Nutzen Sie das Interface **CMSIS_V2**.
3. Stellen Sie die **TOTAL_HEAP_SIZE** in den FreeRTOS-Einstellungen auf mindestens 16384 Bytes (16 KB), um Speicherprobleme beim Anlegen mehrerer Tasks zu vermeiden.
4. Das CubeMX-Tool wird eine Warnung bezüglich der Timebase ausgeben. Ändern Sie unter *System Core* → *SYS* → *Timebase Source* den Timer von **SysTick** auf **TIM6**. Erklären Sie kurz, warum dieser Schritt für die Stabilität des RTOS zwingend notwendig ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Task-Design und Generierung

Legen Sie im Configurator (Reiter *Tasks and Queues*) die folgenden drei Tasks an und generieren Sie anschließend den Code:

1. **HeartbeatTask**: Priorität `osPriorityNormal`
2. **HeavyMathTask**: Priorität `osPriorityBelowNormal`
3. **EmergencyTask**: Priorität `osPriorityHigh` (bleibt in dieser Übung blockiert — siehe Aufgabe 3)

Prüfen Sie nach der Code-Generierung, in welcher Dateistruktur die Tasks in der `main.c` angelegt wurden.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Präemption beweisen

Füllen Sie nun die generierten Task-Skelette mit Leben, um das präemptive Verhalten von FreeRTOS zu testen.

1. Konfigurieren Sie eine LED (z. B. Pin **PE1**) als `GPIO_Output` und weisen Sie sie dem Cortex-M4 zu, falls noch nicht geschehen.
2. Lassen Sie in der `HeartbeatTask` die LED alle 500 ms umschalten. Verwenden Sie **zwingend** `osDelay(500)` und **nicht** `HAL_Delay()`.
3. Programmieren Sie in der `HeavyMathTask` eine reine Endlosschleife, die eine Variable hochzählt (z. B. `volatile uint32_t counter = 0; while(1){ counter++; }`). Verwenden Sie hier **kein** Delay!
4. In der `EmergencyTask` rufen Sie lediglich `osDelay(osWaitForever)`; auf (Task bleibt *Blocked* und verbraucht keine CPU). Ohne diesen Schritt würde die High-Prio-Emergency den Präemptions-Beweis zerstören.
5. Flashen Sie den Code und beobachten Sie die LED. Erklären Sie, warum das System nicht einfriert, obwohl die `HeavyMathTask` 100 % CPU-Last fordert und die CPU niemals freiwillig abgibt.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Starvation (Verhungern) durch Prioritäten

Wir sabotieren nun das System, um die Gefahr falsch gesetzter Prioritäten (RMS-Regelverletzung) zu demonstrieren.

1. Öffnen Sie den CubeMX-Configurator und ändern Sie die Priorität der `HeavyMathTask` auf `osPriorityHigh` (höher als `HeartbeatTask` mit `osPriorityNormal`). Die `EmergencyTask` bleibt blockiert (`osDelay(osWaitForever)`).
2. Generieren und flashen Sie den Code erneut.
3. Beobachten Sie die LED und erklären Sie das Phänomen. Handelt es sich hierbei um eine **Priority Inversion**? Begründen Sie Ihre Antwort.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)